

Assignment 2 - Part 1

Mason Stanley

Task 2: starwars.c

1) Why do the initial and final shield power stay the same?

fork() gives each child its own copy of the parent's memory, so when a child adds to shield_power it only changes its own copy. The parent never changes its copy, so it stays at 50.

2) Why does the order of the output change every time?

The parent and children run at the same time as separate processes, and the OS scheduler decides which one goes first, which can be different each run. wait() only makes the parent wait until they're all done, it doesn't set the order.

Task 4: Timing (N = 100,000,000)

Program	Sequential	1 process	2 processes	3 processes	4 processes
Time taken	0.215 s	0.235 s	0.187 s	0.205 s	0.176 s

What I noticed

Using 1 process was a little slower than sequential because of the extra overhead of forking and the pipe. With more processes it got faster (4 processes was the best at 0.176 s), but it's only about 1.2x faster. That's because filling the array is still done by one process and takes most of the time, so only the adding part gets sped up.

Advantages

- On multiple cores the chunks can be summed at the same time.
- Each process has its own memory, so they can't mess each other up.

Challenges

- Extra overhead from forking and sending results back, so it can be slower.
- Have to use pipes to share results and close all the unused pipe ends.
- More error checking, and splitting the array evenly when N doesn't divide nicely.

Screenshots

Task 1: starwars.c

```
mason@Mason:/mnt/c/Users/mason/OneDrive/Desktop/CSC 410/A2_part1/A2_part1$ ./starwars
Millennium Falcon: Initial shield power level: 50%

Luke: Adjusting shields...
Luke: Shield power level now at 75%
Han: Adjusting shields...
Han: Shield power level now at 70%
Chewbacca: Adjusting shields...
Chewbacca: Shield power level now at 80%
Leia: Adjusting shields...
Leia: Shield power level now at 65%

Final shield power level on the Millennium Falcon: 50%

May the forks be with you!
mason@Mason:/mnt/c/Users/mason/OneDrive/Desktop/CSC 410/A2_part1/A2_part1$
```

Task 3: sequential vs parallel sums (both give the same total)

```
Problems  Output  Debug Console  Terminal  Ports

mason@Mason:/mnt/c/Users/mason/OneDrive/Desktop/CSC 410/A2_part1/A2_part1$ ./seq_sum 100000000
Total sum = 5000000050000000

mason@Mason:/mnt/c/Users/mason/OneDrive/Desktop/CSC 410/A2_part1/A2_part1$ ./parallel_sums 100000000 4
Total sum = 5000000050000000

mason@Mason:/mnt/c/Users/mason/OneDrive/Desktop/CSC 410/A2_part1/A2_part1$
```

Task 4: benchmark.sh timing results

```
Problems  Output  Debug Console  Terminal  Ports

mason@Mason:/mnt/c/Users/mason/OneDrive/Desktop/CSC 410/A2_part1/A2_part1$ bash benchmark.sh
N = 100000000, averaged over 5 runs (wall-clock seconds)
Sequential  1 process    2 processes  3 processes  4 processes
0.215s      0.235s      0.187s      0.205s      0.176s

mason@Mason:/mnt/c/Users/mason/OneDrive/Desktop/CSC 410/A2_part1/A2_part1$
```